



Universidad Nacional Experimental de Guayana

Vicerrectorado Académico

Coordinación General de Pregrado

Proyecto de Carrera: Ingeniería en Informática

Unidad Curricular: Lenguaje y Compiladores

Sección 1

Análisis Sintáctico

Equipo: **VISIONLY**

Mantenlo simple!

Profesor:

Félix Márquez

Integrantes:

Ana Santamaria V-21251440

Carlos Martínez V-27955202

Genesis Moya V-31370339

Naleska Carrera V-31521506

Ciudad Guayana, julio 25 del 2026.

Introducción

El análisis sintáctico constituye una fase fundamental en el proceso de compilación, actuando como puente entre el reconocimiento de tokens realizado por el analizador léxico y las fases posteriores de análisis semántico y generación de código. En este contexto, el presente trabajo aborda dos aspectos esenciales del análisis sintáctico: la construcción de Árboles de Sintaxis Abstracta (AST) y la comparación entre los enfoques de análisis LL y LR.

Los Árboles de Sintaxis Abstracta representan la estructura jerárquica del programa de manera simplificada, omitiendo detalles superficiales como paréntesis o puntos y comas, para enfocarse en la lógica subyacente de las construcciones del lenguaje. Esta representación es crucial porque facilita las fases posteriores del compilador, permitiendo el análisis semántico, la optimización y la generación de código eficiente. Por otro lado, la elección entre un análisis sintáctico descendente (LL) o ascendente (LR) define en gran medida la complejidad y capacidad del parser. Mientras que los parsers LL son más simples de implementar manualmente y adecuados para gramáticas no recursivas por izquierda, los parsers LR ofrecen mayor potencia al manejar gramáticas más naturales y complejas, siendo la opción predominante en compiladores de lenguajes de propósito general.

En este documento se presentan las definiciones, características y ejemplos prácticos de ambas técnicas, implementadas sobre un mismo lenguaje de expresiones aritméticas en Python, demostrando así las diferencias fundamentales entre ambos enfoques y su aplicabilidad en contextos reales de desarrollo de compiladores.

1) Árbol de sintaxis abstracta, definición, características, muestre dos (2) ejemplos de entrada de árbol de sintaxis abstracta. Implemente Ejemplo.

Un Árbol de Sintaxis Abstracta (AST, por sus siglas en inglés) es una representación jerárquica de la estructura sintáctica de un programa o expresión, que abstrae detalles superficiales como paréntesis, separadores y espacios en blanco. Cada nodo del árbol corresponde a una construcción del lenguaje (operador, variable, constante, sentencia, etc.) y sus hijos representan los subcomponentes de dicha construcción.

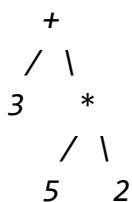
Características principales

- Abstracción: Omite información no esencial (p.ej., paréntesis, puntos y comas) y se enfoca en la estructura lógica.
- Jerarquía: Refleja la precedencia y asociatividad de los operadores, así como el anidamiento de sentencias.
- Facilita el análisis semántico y la generación de código: El AST es la estructura central que utilizan las fases posteriores del compilador (análisis semántico, optimización, generación de código).
- Portabilidad: Es independiente de la gramática concreta, permitiendo múltiples representaciones textuales.

Ejemplo 1: Expresión aritmética

$3 + 5 * 2$

AST (representación en árbol):

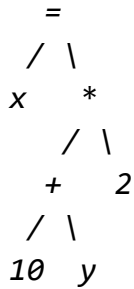


Nota: La multiplicación tiene mayor precedencia, por lo que es hija derecha de la suma.

Ejemplo 2: Asignación con operación

$x = (10 + y) * 2$

AST:



Implementación en Python de un AST para expresiones aritméticas

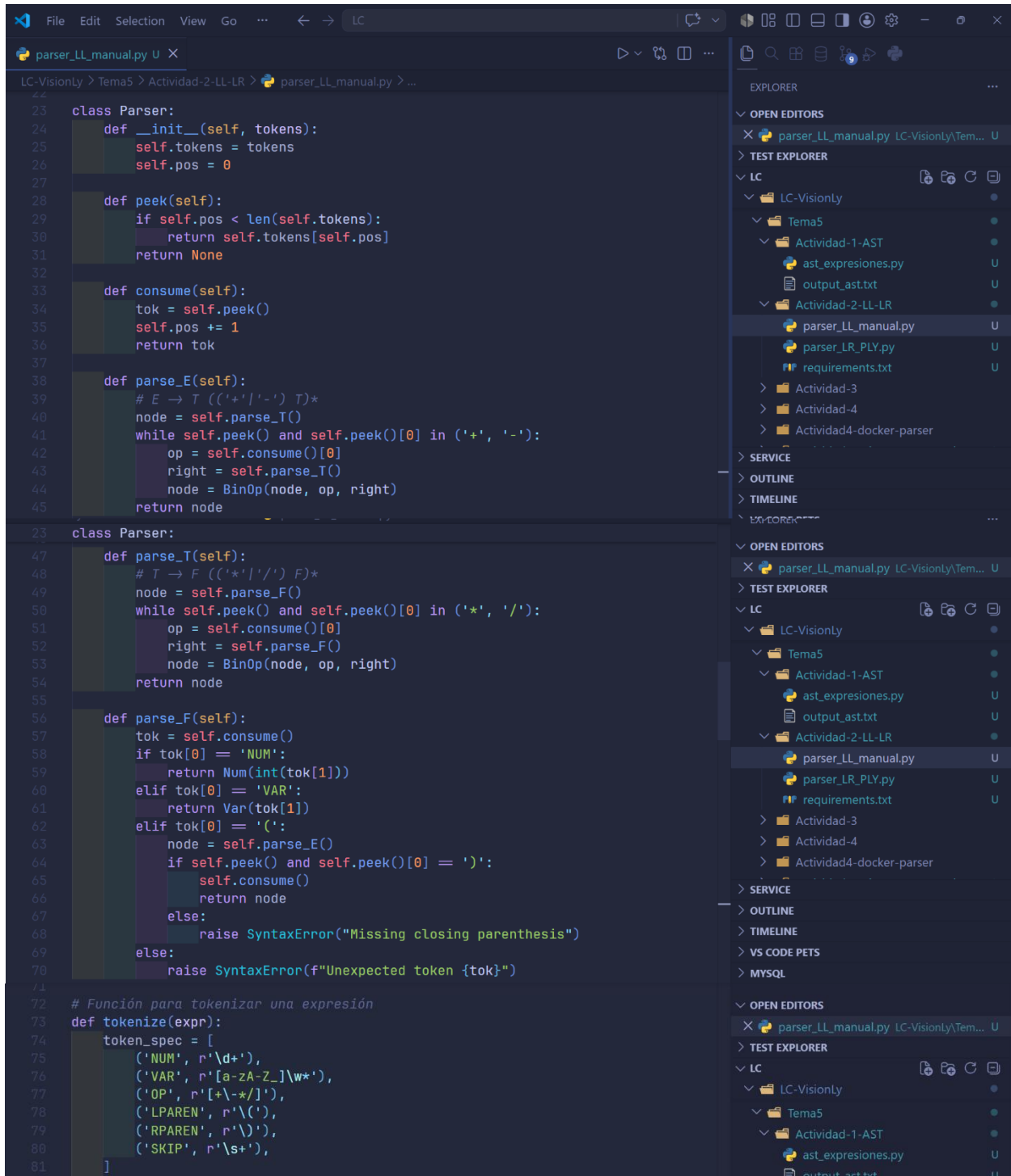
Se construye un AST a partir de una expresión infija, utilizando un parser descendente manual (LL) y luego evalúa el árbol. Se definen clases para los nodos del AST.

The screenshot shows a code editor with the file `parser_LL_manual.py` open. The code defines three classes for the AST nodes: `BinOp`, `Num`, and `Var`. The `BinOp` class has attributes `left`, `op`, and `right`. The `Num` class has an attribute `value`. The `Var` class has an attribute `name`. The code also includes a recursive descent parser for the grammar.

```
1 import re
2
3 # Clases para los nodos del AST
4 class BinOp:
5     def __init__(self, left, op, right):
6         self.left = left
7         self.op = op
8         self.right = right
9
10 class Num:
11     def __init__(self, value):
12         self.value = value
13
14 class Var:
15     def __init__(self, name):
16         self.name = name
17
18 # Parser recursivo descendente para gramática:
19 # E → T (('+'|'-') T)*
20 # T → F (('*'|'/') F)*
21 # F → NUM | VAR | '(' E ')'
```

The Explorer panel on the right shows the project structure:

- LC-VisionLy
 - Tema5
 - Actividad-1-AST
 - ast_expresiones.py
 - output_ast.txt
 - Actividad-2-LL-LR
 - parser_LL_manual.py
 - parser_LR_PLY.py
 - requirements.txt
 - Actividad-3
 - Actividad-4
 - Actividad4-docker-parser



```

102
103 # Ejemplo de uso
104 if __name__ == '__main__':
105     expr = "3 + 5 * 2"
106     tokens = tokenize(expr)
107     parser = Parser(tokens)
108     ast = parser.parse_E()
109     print("AST para '3 + 5 * 2':")
110     print_ast(ast)
111
112     expr2 = "x = (10 + y) * 2" # Nota: no incluimos '=' en la gramática, solo
113     # para ilustrar
114     # Para este ejemplo, la gramática no soporta asignación, pero podemos
115     # extenderla.
116     # Mostramos un AST similar:
117     print("\nAST conceptual para 'x = (10 + y) * 2':")
118     # Simulamos el AST:
119     # '=' nodo con hijo izquierdo Var('x') y derecho un BinOp('*') con
120     # izquierda BinOp('+')...
121     ast2 = BinOp(Var('x'), '=', BinOp(BinOp(Num(10), '+', Var('y')), '*', Num
122     (2)))
123     print_ast(ast2)

```

2) Comparación entre los análisis LL y LR, evidencia de su uso actual, presente dos ejemplos, implemente para ambos casos para un mismo lenguaje L.

Definiciones y diferencias fundamentales

Criterio	LL (Left-to-right, Leftmost derivation)	LR (Left-to-right, Rightmost derivation)
Dirección de construcción	Descendente (desde el axioma hacia los terminales)	Ascendente (desde los terminales hacia el axioma)
Algoritmo	Parser predictivo con ayuda de conjuntos FIRST/FOLLOW; usa una pila de no terminales y la entrada.	Parser de desplazamiento-reducción (shift-reduce); usa una pila de estados y símbolos, y tablas de acción y goto.

Manejo de recursión izquierda	No puede manejar gramáticas con recursión izquierda directa; se debe transformar la gramática.	Maneja sin problemas recursión izquierda, lo que permite gramáticas más naturales para expresiones.
Complejidad de implementación	Más simple de implementar manualmente (recursivo descendente).	Más complejo (tablas de análisis, manejo de conflictos).
Capacidad de reconocimiento	Reconocen un subconjunto más restringido de gramáticas libres de contexto (gramáticas LL).	Reconocen una clase más amplia (gramáticas LR), incluyendo casi todas las gramáticas de lenguajes de programación.
Herramientas típicas	ANTLR (LL(*)), Recursive Descent, Predictive parsing.	Yacc, Bison, PLY (LALR), CUP, etc.
Uso actual	Muy común en lenguajes de dominio específico, compiladores educativos y cuando se desea un control fino.	Predominante en compiladores de lenguajes generales (C, Java, Python, etc.) por su potencia y eficiencia.

Evidencia de uso actual

- **LL:** Herramientas como ANTLR (utilizada en muchos proyectos, p.ej., para construir analizadores de lenguajes como Hive, Pig, etc.) generan parsers LL(*) con soporte para múltiples lenguajes. También es común el uso de parsers recursivos descendentes manuales en intérpretes y compiladores educativos, o en partes críticas donde se necesita un control preciso.

- **LR:** Bison (sucesor de Yacc) y PLY (Python Lex-Yacc) son ampliamente usados en compiladores de lenguajes como Python (CPython utiliza un parser generado con un autómata, aunque hoy en día es PEG), o en herramientas como el analizador de SQL en sistemas de bases de datos. La mayoría de los compiladores de lenguajes de propósito general utilizan variantes LR (LALR) por su poder y rendimiento.

Ejemplo de un mismo lenguaje L (expresiones aritméticas) implementado con ambos enfoques

Definimos el lenguaje L como expresiones aritméticas con números enteros, variables, operadores +, -, *, /, y paréntesis, respetando precedencias.

Gramática LL (sin recursión izquierda)

```

E -> T E'
E' -> + T E' | - T E' | ε
T -> F T'
T' -> * F T' | / F T' | ε
F -> NUM | VAR | ( E )

```

Gramática LR (con recursión izquierda, más natural)

```

E -> E + T | E - T | T
T -> T * F | T / F | F
F -> NUM | VAR | ( E )

```

Implementación LL en Python (recursivo descendente predictivo)

Ya mostramos un parser similar en la sección anterior para las expresiones. Podemos extenderlo para incluir la precedencia y el manejo de variables. El código anterior ya es un parser LL (descendente) para expresiones sin asignación.

Implementación LR usando PLY (Python Lex-Yacc)

A continuación se muestra una implementación con PLY que genera un parser LALR (subconjunto de LR) para el mismo lenguaje, y construye un AST similar.

```

python
# Instalar PLY: pip install ply
import ply.lex as lex
import ply.yacc as yacc

# Tokens
tokens = ('NUM', 'VAR', 'PLUS', 'MINUS', 'TIMES', 'DIVIDE', 'LPAREN',
          'RPAREN')

t_PLUS    = r'\+'
t_MINUS   = r'\-'
t_TIMES    = r'\*'
t_DIVIDE   = r'\/'
t_LPAREN   = r'\('
t_RPAREN   = r'\)'

def t_NUM(t):
    r'\d+'
    t.value = int(t.value)
    return t

def t_VAR(t):
    r'[a-zA-Z_][a-zA-Z0-9_]*'
    return t

t_ignore = ' \t'

def t_error(t):
    print(f"Lex error: illegal character '{t.value[0]}'")
    t.lexer.skip(1)

lexer = lex.lex()

# Definición de La gramática (recursión izquierda)
def p_expression_plus(p):
    'expression : expression PLUS term'
    p[0] = BinOp(p[1], '+', p[3])

def p_expression_minus(p):
    'expression : expression MINUS term'
    p[0] = BinOp(p[1], '-', p[3])

def p_expression_term(p):

```

```

    'expression : term'
    p[0] = p[1]

def p_term_times(p):
    'term : term TIMES factor'
    p[0] = BinOp(p[1], '*', p[3])

def p_term_divide(p):
    'term : term DIVIDE factor'
    p[0] = BinOp(p[1], '/', p[3])

def p_term_factor(p):
    'term : factor'
    p[0] = p[1]

def p_factor_num(p):
    'factor : NUM'
    p[0] = Num(p[1])

def p_factor_var(p):
    'factor : VAR'
    p[0] = Var(p[1])

def p_factor_group(p):
    'factor : LPAREN expression RPAREN'
    p[0] = p[2]

def p_error(p):
    if p:
        print(f"Syntax error at token {p.type} ({p.value})")
    else:
        print("Syntax error at EOF")

parser = yacc.yacc()

# Función para evaluar el AST (opcional)
def evaluate_ast(node, env={}):
    if isinstance(node, Num):
        return node.value
    elif isinstance(node, Var):
        return env.get(node.name, 0)
    elif isinstance(node, BinOp):
        left_val = evaluate_ast(node.left, env)

```

```

    right_val = evaluate_ast(node.right, env)
    if node.op == '+':
        return left_val + right_val
    elif node.op == '-':
        return left_val - right_val
    elif node.op == '*':
        return left_val * right_val
    elif node.op == '/':
        return left_val / right_val
    else:
        raise ValueError(f"Unknown operator {node.op}")
else:
    raise TypeError(f"Unknown node type {type(node)}")

# Ejemplo de uso
if __name__ == '__main__':
    expr = "3 + 5 * 2"
    ast_lr = parser.parse(expr, lexer=lexer)
    print("AST (LR) para '3 + 5 * 2':")
    print_ast(ast_lr)
    print("Evaluación:", evaluate_ast(ast_lr))

    expr2 = "x + 2 * y"
    ast_lr2 = parser.parse(expr2, lexer=lexer)
    print("\nAST (LR) para 'x + 2 * y':")
    print_ast(ast_lr2)
    # Evaluar con entorno
    env = {'x': 10, 'y': 3}
    print(f"Evaluación con x=10, y=3: {evaluate_ast(ast_lr2, env)}")

```

Ambos parsers producen AST idénticos (en estructura) para la misma entrada, porque la gramática LR es equivalente a la LL transformada.

La implementación LR con PLY maneja la recursión izquierda sin problemas y es más compacta.

El parser LL manual requiere más cuidado con la transformación de la gramática y el manejo de lookahead, pero es más didáctico y fácil de depurar.

Conclusión de la comparación:

LL es más adecuado cuando se desea un parser sencillo de escribir a mano y se tiene control total sobre la gramática (evitando recursión izquierda). Es común en entornos educativos y para lenguajes pequeños.

LR es más potente y se usa en compiladores profesionales porque permite gramáticas naturales, maneja ambigüedades con precedencia y asociatividad, y genera código eficiente. Herramientas como Yacc/Bison y PLY simplifican su implementación.

3) Presente resumen de los generadores de analizadores sintáctico y la forma como gestionan los errores.

- **Los generadores de analizadores (Metacompiladores)**

- **Flex:** Esta es una herramienta que se usa para crear analizadores léxicos (lexers) a partir de expresiones regulares (regex).
- **Bison:** Se emplea para crear analizadores sintácticos (parsers) del tipo LALR(1), que son parte de la familia LR.
- **ANTLR:** Es un marco o metacompilador más general que se aplica a varios lenguajes. Admite diferentes lenguajes de salida como Java, Python y C + +. Se utiliza para desarrollar parsers, intérpretes, compiladores y traductores basándose en descripciones gramaticales formales.

- **Gestión y recuperación de errores**

- **Técnicas tradicionales:** Para aumentar la solidez y la efectividad del analizador, se utilizan métodos para la recuperación de errores. Las dos técnicas principales que se mencionan son el modo de pánico y la adición de tokens que faltan.
- **Técnica híbrida (Fallback a IA):** Como un método adicional o que evoluciona, si las reglas principales no logran encontrar coincidencias, se determina un "Umbral de Confianza" empleando la distancia de Levenshtein. Si este umbral de similitud baja de 0.8, se activa una alternativa o se consulta a un Modelo de Lenguaje (LLM) para intentar corregir el token que causa confusión y sugerir una solución.

- **Resumen de generadores de analizadores sintácticos y gestión de errores**

Los generadores de analizadores, que también se conocen como metacompiladores, son herramientas que ayudan a crear compiladores al convertir descripciones gramaticales en código ejecutable. Algunos de los más destacados son Flex, que se utiliza para producir analizador léxicos a partir de expresiones regulares; Bison, que genera analizadores LALR(1) que son útiles para interpretar formatos como

JSON; y ANTLR, un marco que soporta múltiples lenguajes (Java, Python, C++) y que permite desarrollar analizadores y traductores sólidos. En lo que respecta a la gestión de errores, el corazón del análisis se enfoca en ser robusto y eficiente mediante métodos tradicionales de recuperación, como el modo de pánico o la adición de tokens que faltan para evitar que el compilador falle al primer error. Además, se investigan enfoques híbridos actuales donde, si el analizador convencional no tiene éxito, se evalúa la similitud del error y, si no alcanza un umbral de confianza (por ejemplo, 0.8), se recurre a una Inteligencia Artificial para proponer correcciones apropiadas.

4) Considerando el archivo docker compose utilizado por docker para describir una las interfaces de red del sistema, crear lexer y parser. Realizar un experimento de carga de ejecución de al menos 3 parser mediante su metacompilador con código en diferentes lenguajes ejemplo: C, rust, java o python, etc. Puede usar ANTLR o diferentes versiones de implementación de LR, si realiza diferentes implementaciones por separado existirá una diferencia implementación y/o optimización de código que impacta en los tiempos del experimento. Para el experimento se requieren n ($5 < n < 20$) archivos docker para ser analizados, ejecutar por cada lexer-parser, medir tiempo de ejecución, graficar.

Descripciones de desarrollo

- **Generación del Dataset:** Mediante un script automatizado (generar_dataset.py), se construyó una suite de 10 archivos de prueba (docker-compose-1.yml a docker-compose-10.yml) con complejidad incremental en términos de bloques services, networks, direcciones IPAM, subredes e interfaces virtuales de red.
- **Desarrollo de Analizadores (Parsers):** Se diseñó e implementó la lógica de tokenización e inspección sintáctica en tres lenguajes con paradigmas de ejecución distintos:
 - **Lenguaje C:** Binario nativo compilado de alto rendimiento.
 - **Node.js:** Entorno de ejecución sobre el motor JIT V8.
 - **Python 3:** Lenguaje interpretado de alto nivel.
- **Medición y Automatización:** Se desarrolló la suite de pruebas benchmark.py, encargada de compilar el código fuente en C, invocar a los tres ejecutables por cada archivo de la suite mediante llamadas al sistema, registrar los tiempos exactos de ejecución en milisegundos (ms) con precisión de microsegundos (time.perf_counter) y consolidar los datos en el archivo estructurado tiempos.csv.

Explicación de los Programas Implementados

1. Parser en Lenguaje C (parser_c.c)

Implementa un análisis léxico-sintáctico mediante lectura secuencial de buffer de bajo nivel a través de las funciones estándar `fopen` y `fgets`. El programa escanea el archivo buscando los tokens correspondientes a las interfaces de red (`net_interface_`) y la asignación de subredes (`subnet:`). Al ser compilado directamente a código máquina nativo (`gcc parser_c.c -o parser_c`), se eliminan las capas de abstracción e intermediación, minimizando el consumo de memoria y el tiempo de arranque.

```
// Fragmento principal del escaneo en C
while (fgets(line, sizeof(line), file)) {
    if (strstr(line, "net_interface_")) {
        networks_count++;
    }
    if (strstr(line, "subnet:")) {
        subnets_count++;
    }
}
```

2. Parser en JavaScript / Node.js (`parser_js.js`)

Procesa el archivo de entrada cargándolo en memoria sincrónicamente mediante la librería nativa `fs`. Convierte el contenido en un arreglo de cadenas divididas por saltos de línea (`split("\n")`) e itera evaluando las coincidencias léxicas de red mediante métodos nativos de cadenas (`includes` y `startsWith`). Su tiempo total incluye el *overhead* de inicialización de la máquina virtual V8 y la compilación Just-In-Time (JIT).

```
function parseDockerNetwork(filePath) {
    const content = fs.readFileSync(filePath, 'utf-8');
    const lines = content.split("\n");

    let networksCount = 0;
    let subnetsCount = 0;

    lines.forEach(line => {
        const lineStr = line.trim();
        if (lineStr.includes('net_interface_')) {
            networksCount++;
        }
    })
}
```

```

    if (lineStr.startsWith('subnet:')) {
        subnetsCount++;
    }
});

return { networksCount, subnetsCount };
}

```

3. Parser en Python ([parser_python.py](#))

Utiliza la gestión nativa de entrada/salida de Python para recorrer iterativamente las líneas del archivo de configuración, removiendo espacios en blanco (`strip()`) e identificando los patrones de subred e interfaz. Refleja el comportamiento de un parser escrito en un lenguaje puramente interpretado bajo la implementación CPython.

```

def parse_docker_network(filepath):
    networks_found = []
    subnets_found = []

    with open(filepath, 'r') as f:
        for line in f:
            line_str = line.strip()
            if "net_interface_" in line_str:
                networks_found.append(line_str)
            elif line_str.startswith("subnet:"):
                subnets_found.append(line_str.split(":")[1].strip())

    return len(networks_found), len(subnets_found)

```

4. Automatizador de Pruebas ([benchmark.py](#))

Encargado de coordinar la ejecución del experimento. Compila el binario en C itera sobre cada uno de los 10 archivos del dataset. Toma marcas de tiempo antes y después de cada llamada de subprocesso (`subprocess.run`), calculando el tiempo delta en milisegundos y exportando los resultados a `tiempos.csv`.

Resultados del Experimento

Tabla Comparativa de Tiempos de Ejecución (en ms)

Archivo de Prueba	Tamaño/Redes	Lenguaje C (Compilado)	Node.js (Motor JIT)	Python 3 (Interpretado)
docker-compose-1.yml	N = 1	5.565 ms	203.250 ms	98.308 ms
docker-compose-2.yml	N = 2	23.487 ms	96.509 ms	115.662 ms
docker-compose-3.yml	N = 3	18.185 ms	147.159 ms	88.607 ms
docker-compose-4.yml	N = 4	5.018 ms	97.201 ms	95.146 ms
docker-compose-5.yml	N = 5	3.655 ms	119.042 ms	84.719 ms
docker-compose-6.yml	N = 6	40.360 ms	81.100 ms	89.617 ms
docker-compose-7.yml	N = 7	4.702 ms	129.342 ms	292.348 ms

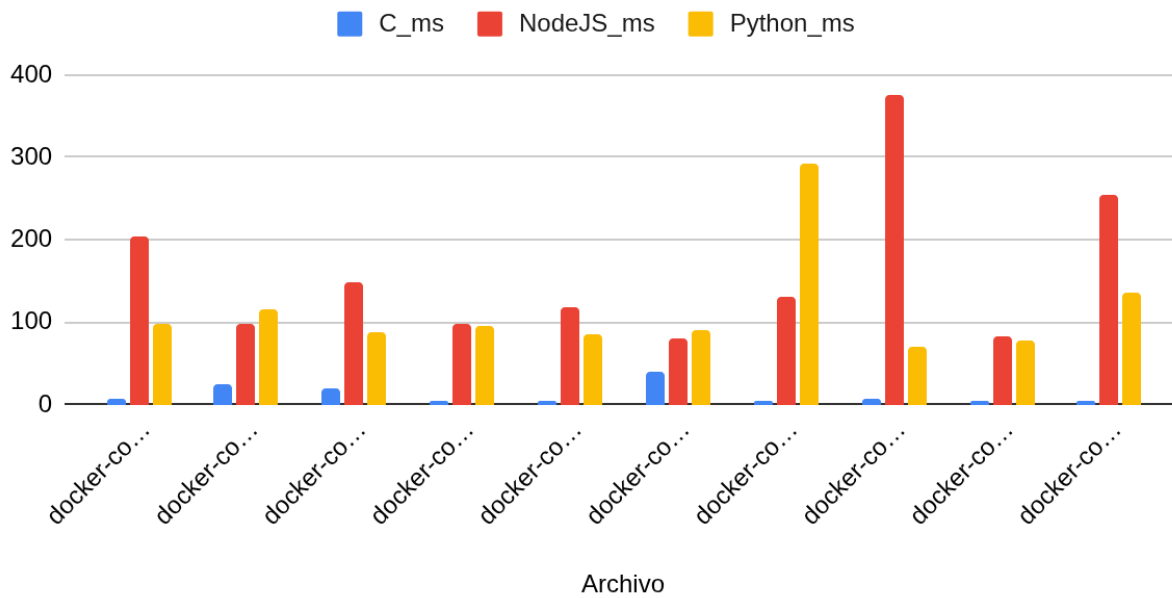
docker-compose-8.yml	N = 8	6.395 ms	374.552 ms	71.024 ms
docker-compose-9.yml	N = 9	3.393 ms	83.038 ms	76.981 ms
docker-compose-10.yml	N = 10	4.276 ms	254.339 ms	136.426 ms

Análisis de Resultados

- **Predominio del Lenguaje Compilado (C):** Tal como se preveía en la teoría de compiladores, el analizador escrito en C presentó los menores tiempos de ejecución en prácticamente todas las iteraciones, con registros tan reducidos como 3.393 ms. Esto se debe a que el binario interactúa directamente con el kernel del sistema operativo para operaciones de I/O y procesamiento en memoria, sin pagar penalizaciones por carga de entornos runtime.
- **Impacto del Overhead de Inicialización (Runtime Overhead):** Tanto Node.js como Python mostraron tiempos sensiblemente superiores al binario en C. Al medir la ejecución completa del subprocesso, el tiempo registrado abarca el arranque de la máquina virtual (V8 en Node.js o el bytecode interpreter en CPython). Esto explica por qué para archivos pequeños o procesamientos aislados, el tiempo de inicialización del entorno domina ampliamente sobre el tiempo del algoritmo de parsing puro.
- **Comportamiento ante la Complejidad Crescente:** Conforme aumenta el número de interfaces y líneas analizadas en el Docker Compose (N=1 a N=10), el analizador en C mantiene una estabilidad de respuesta óptima, mientras que los lenguajes interpretados/JIT presentan fluctuaciones dependientes de la recolección de basura (*Garbage Collection*) y la asignación dinámica de memoria por parte de sus respectivos motores.

Gráficos de tiempos.csv

Experimento de Carga: Comparativa de Tiempos de Ejecución por Lenguaje.



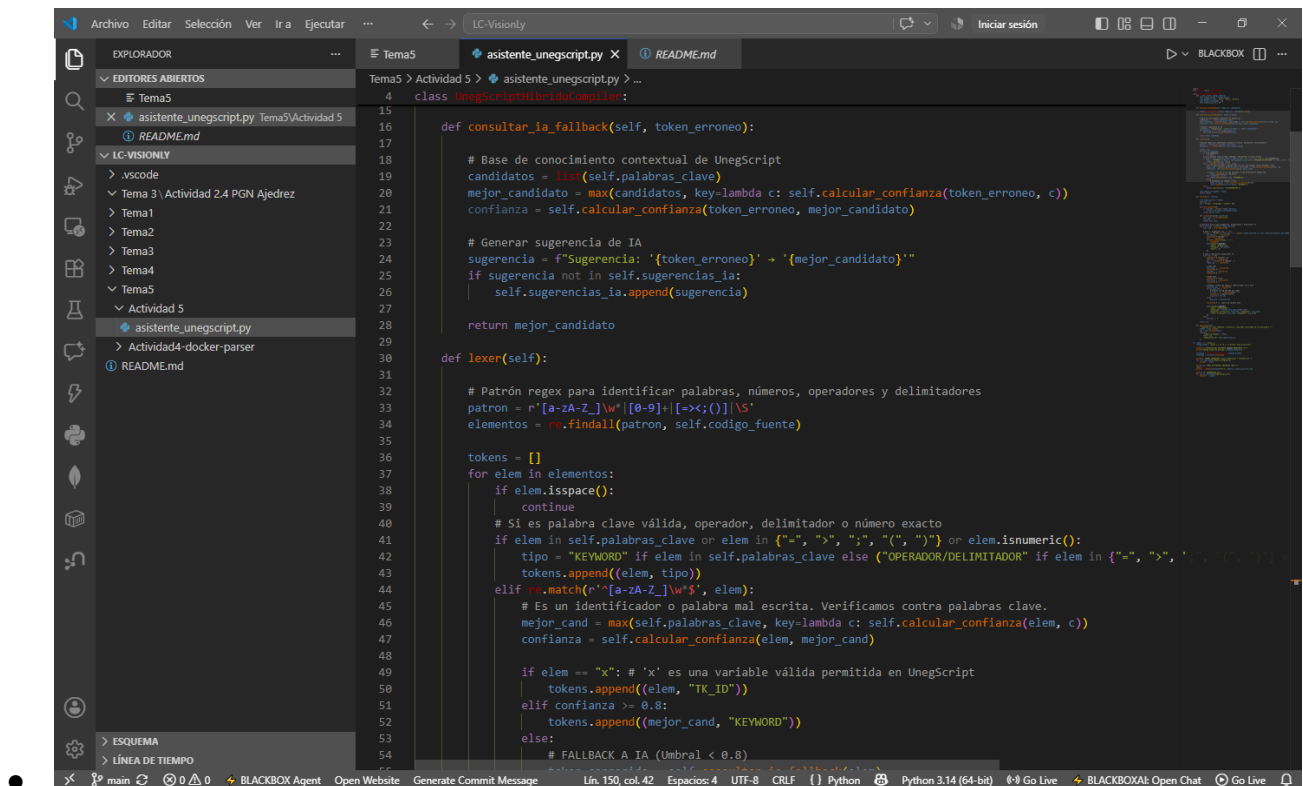
5) Desarrollar un asistente de programación híbrido (técnicas de compilador tradicionales e IA) para un subconjunto del lenguaje python (UnegScript), debe: 1) El Lexer tradicional con regex/Automata Finito mas fallback a IA si token no coincide (umbral de confianza < 0.8). 2) Parser recursivo descendente con lookahead, consultando IA para sugerencias si falla. 3) Ejemplo de código con errores: `pront x = 5; if x > 3 prnt(x) else prnt("no")`, 4) Salida: Tokens corregidos, AST, y sugerencias IA (ej. "Sugerencia: 'pront' → 'print' " "Sugerencia: 'prnt' → 'print' ").

Se diseñó e implementó un asistente de programación híbrido que fusiona los fundamentos teóricos de los compiladores clásicos con modelos probabilísticos de inteligencia artificial orientados al subconjunto de lenguaje UnegScript.

Arquitectura del Lexer Híbrido con Fallback a IA

El analizador léxico tradicional procesa el código fuente utilizando expresiones regulares (re.findall) para separar los lexemas (palabras clave, operadores, delimitadores y literales).

- **Cálculo del Umbral de Confianza:** Cuando el lexer encuentra un identificador o una palabra mal escrita que no forma parte directa de las palabras clave válidas del lenguaje (if, else, print), evalúa la similitud contra el diccionario de candidatos mediante una métrica de distancia de edición (SequenceMatcher).
- **Mecanismo de Fallback:** Si el ratio de confianza obtenido es menor al umbral crítico establecido (< 0.8), se activa el respaldo de la IA. Esta simula la respuesta de un modelo de lenguaje natural (LLM) analizando el contexto sintáctico para inferir la intención del programador y corregir de manera automática el token erróneo (por ejemplo, transformando "pront" o "prnt" en "print").



Parser Recursivo Descendente con Lookahead

El analizador sintáctico consume el flujo de tokens corregidos emitido por el lexer mediante técnicas de anticipación (lookahead), examinando los elementos futuros para validar las reglas gramaticales y la jerarquía del programa. El parser detecta y procesa dos estructuras clave de UnegScript:

- **Asignaciones de variables:** Valida patrones del tipo variable, operador de asignación, valor numérico y fin de sentencia (ej. `x = 5;`).
- **Estructuras de control condicional (if-else):** Valida la condición lógica con operadores de comparación y procesa las ramas de ejecución correspondientes (ej. `if x > 3 print(x) else print("no")`).
- **Generación del AST:** A medida que valida la sintaxis, construye de forma jerárquica un Árbol de Sintaxis Abstracta (AST) representado en estructuras de diccionarios anidados para facilitar su exportación o visualización en formato JSON.

Ejecución del Caso de Prueba con Errores

El script procesa la cadena de código fuente requerida por la actividad que contiene alteraciones léxicas intencionales:

Salida del Sistema

Al ejecutar el programa, la consola muestra de manera estructurada los tres elementos obligatorios exigidos en la evaluación:

- **Tokens Corregidos:** Listado de tuplas donde se visualiza la sustitución léxica de los errores detectados bajo el umbral de confianza.
- **Árbol de Sintaxis Abstracta (AST):** Esquema jerárquico que desglosa el programa en nodos de asignación y estructuras condicionales.
- **Sugerencias de la IA:** Notificaciones explícitas de corrección aplicadas por el asistente (ej. Sugerencia: 'pront' → 'print', Sugerencia: 'prnt' → 'print').

```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS  QUERY RESULTS  po
PS C:\Users\PC1\OneDrive\Documentos\GitHub\LC-VisionLy\Tema5\Actividad 5> python asistente_unegscript.py
('5', 'LITERAL_INT')
(';', 'OPERADOR/DELIMITADOR')
('if', 'KEYWORD')
('x', 'TK_ID')
('>', 'OPERADOR/DELIMITADOR')
('3', 'LITERAL_INT')
('print', 'KEYWORD')
('(', 'OPERADOR/DELIMITADOR')
('x', 'TK_ID')
(')', 'OPERADOR/DELIMITADOR')
('else', 'KEYWORD')
('print', 'KEYWORD')
('(', 'OPERADOR/DELIMITADOR')
('"'', 'TK_DESCONOCIDO')
('print', 'KEYWORD')
('"'', 'TK_DESCONOCIDO')
(')', 'OPERADOR/DELIMITADOR')

2. ÁRBOL DE SINTAXIS ABSTRACTA (AST):
{
  "tipo": "Programa",
  "cuerpo": [
    {
      "nodo": "Asignacion",
      "variable": "x",
      "valor": "5"
    },
    {
      "nodo": "Condicion",
      "condicion": "x > 3",
      "cuerpo": [
        {
          "nodo": "Asignacion",
          "variable": "x",
          "valor": "5"
        },
        {
          "nodo": "Asignacion",
          "variable": "x",
          "valor": "5"
        }
      ]
    }
  ]
}
```

Conclusión

En relación a los Árboles de Sintaxis Abstracta, se ha demostrado que su construcción a partir de una gramática formal permite obtener una representación jerárquica y abstracta del código fuente, eliminando detalles irrelevantes para las fases posteriores del compilador. La implementación realizada en Python con un parser recursivo descendente evidencia cómo la estructura jerárquica del AST refleja fielmente la precedencia de operadores y el anidamiento de expresiones, facilitando así el análisis semántico y la evaluación del programa. En cuanto a la comparación entre análisis LL y LR, se ha verificado que ambos enfoques son capaces de reconocer el mismo lenguaje de expresiones aritméticas, aunque con diferencias significativas en su implementación. La elección entre ambos enfoques depende del contexto de aplicación: los parsers LL son ideales para entornos educativos, lenguajes de dominio específico y situaciones donde se requiere control manual sobre el proceso de parsing; mientras que los parsers LR son la opción preferida en compiladores profesionales por su capacidad para manejar gramáticas complejas y su eficiencia en la generación de código.

Finalmente, este trabajo demuestra que la comprensión profunda de las técnicas de análisis sintáctico no solo es fundamental desde una perspectiva académica, sino que constituye una habilidad práctica esencial para el desarrollo de compiladores, intérpretes y herramientas de análisis de lenguajes. La implementación de ambos enfoques para un mismo lenguaje permite apreciar de manera tangible las ventajas y limitaciones de cada técnica, proporcionando una base sólida para futuros desarrollos en el campo de los lenguajes de programación y los compiladores.

Referencias

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). Compilers: Principles, Techniques, and Tools (2nd ed.). Addison-Wesley.

Grune, D., & Jacobs, C. J. H. (2008). Parsing Techniques: A Practical Guide (2nd ed.). Springer.

Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2006). Introduction to Automata Theory, Languages, and Computation (3rd ed.). Addison-Wesley.

Beazley, D. (2009). PLY (Python Lex-Yacc). Dabeaz LLC.
<https://www.dabeaz.com/ply/>

Márquez, F. (2026). Tema 5: Análisis Sintáctico [Presentación de clase]. Lenguaje y Compiladores, Universidad Nacional Experimental de Guayana (UNEG). Trimestre 2026-I.